

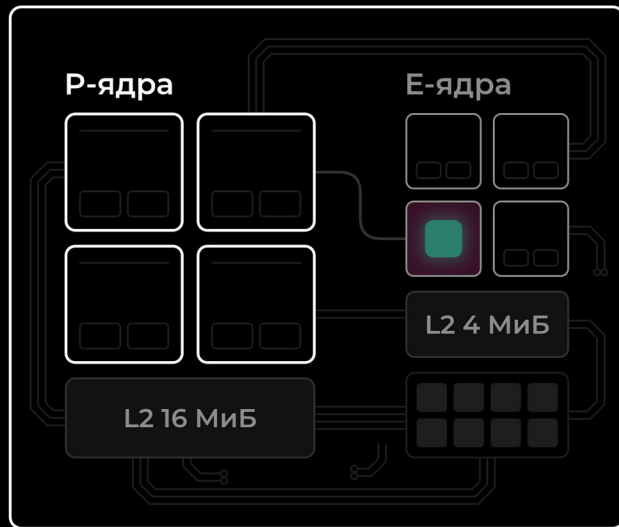
Алгоритмы всё ещё нужны?

$O(n)$ ПРОИГРАЛ $O(n \log n)$

Ваш алгоритм не выбирал
процессор, на котором работает

Никита Нагорнов

iOS, компания «Исходный код»





НИКИТА НАГОРНОВ

iOS · «Исходный код»

Алгоритмы. Компиляторы. Айфоны.

КОМУ ПРИГОДИТСЯ ЭТОТ ДОКЛАД



Разработчикам

Выбираете алгоритмы
и контейнеры

Тимлидам и ревьюерам

Проверяете перф-изменения
на ревью

Авторам перф-тестов

Мерите в CI
и на целевом устройстве

КОД-РЕВЬЮ, ВТОРНИК, 17:40



```
// dedup ids after offline sync
- std::sort(ids.begin(), ids.end());
- ids.erase(std::unique(ids.begin(), ids.end()), ids.end());
+ std::unordered_set<uint64_t> s(ids.begin(), ids.end());
+ ids.assign(s.begin(), s.end());
```

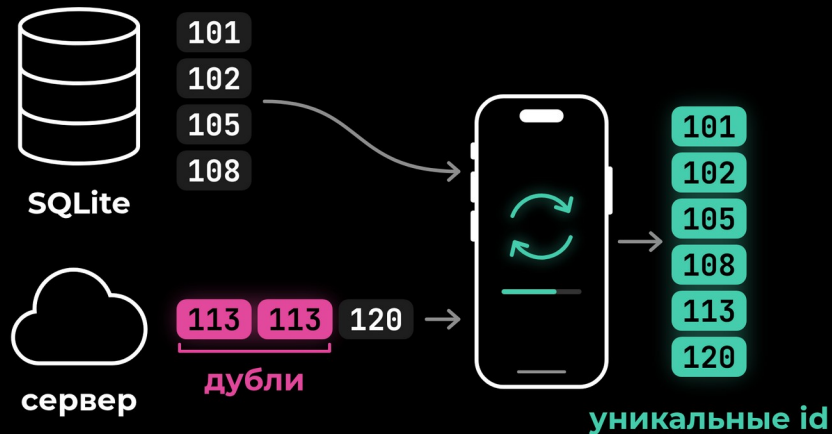
Коллега: «**Линейно вместо $n \log n$** »

Approve ✓



С ЧЕГО ВСЁ НАЧАЛОСЬ

ЗАДАЧА: СКЛЕИТЬ ЛОКАЛЬНЫЕ id И ДЕЛЬТУ С СЕРВЕРА БЕЗ ПОВТОРОВ



1 Локальные id отсортированы
и уникальны

2 Ключ — uint64_t

3 QoS utility

«не срочно»

ВАША СТАВКА

$O(N \log N)$ ПРОТИВ СРЕДНЕГО $O(N)$



A

sort + unique

$O(N \log N)$

B

unordered_set

$O(N)$ в среднем

C

сначала спросил бы
условие

$\log_2 10^6 \approx 20$

1M 64-битных id · все
разные · один поток ·
порядок не важен

Что первым отправим на замер?

$O(n)$ ПРОТИВ $O(n \log n)$



Соперников объявили. **Взвешивание пропустили**

СТАВКА №1 · ЧТО ПОКАЗАЛ СЕКУНДОМЕР

ПЕРВЫЙ ЗАМЕР



нс на элемент · $N = U = 2^{20}$ · M2 Pro · libc++ · меньше — лучше

sort + unique



19,9

×2,6

unordered_set



51,3

Четыре раунда: память, реализация, данные, процессор

M2 Pro: эта серия Xeon VM: отдельная серия iPhone 14 Pro: не измерено

ГДЕ ИЗМЕРЯЕМ

ТРИ СРЕДЫ, ОДНА ЗАДАЧА



M2 Pro

macOS, libc++
Ноутбук разработчика

Xeon VM

Linux, libstdc++ и libc++
Облачная виртуальная машина

iPhone 14 Pro

A16 Bionic, libc++
userInitiated и utility

В каждой серии фиксируем N, U, контракт и режим

ЛИНЕЙНО. ПРОСТО ОЧЕНЬ ТЯЖЕЛО

HA
RD fest



ПРАВИЛО: В ТАЙМЕРЕ ВСЯ ОПЕРАЦИЯ



```
auto v = input;           // вне таймера  
timer.start(); alg(v); timer.stop();  
if (sorted(v) != ref) abort(); // и в релизе
```

в таймере:

reserve

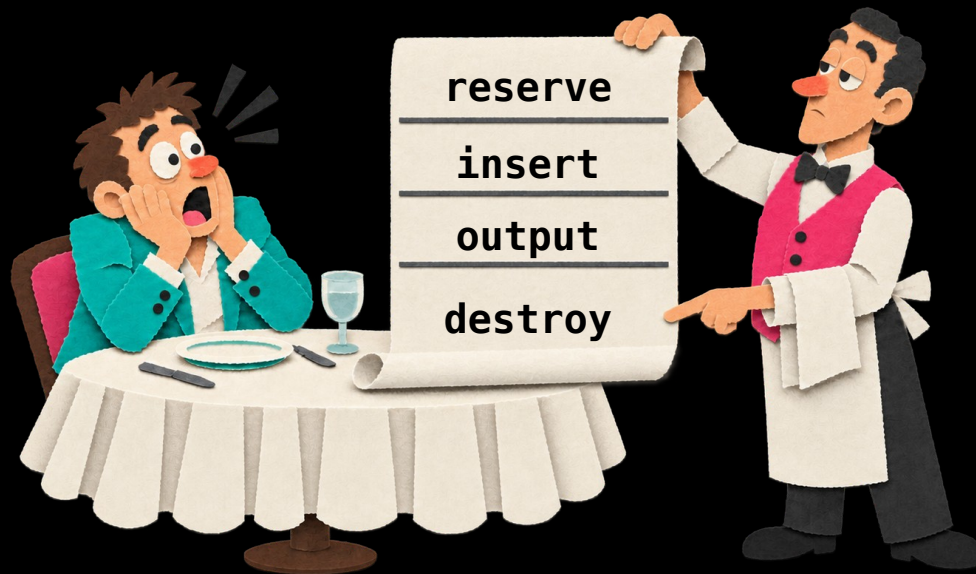
вставки

выгрузка

деструктор

**Быстрое, но неправильное решение
в турнире не участвует**

А ЗАКРЫВАЮЩАЯ СКОБКА?



Вы же ещё **закрывающую скобку** заказывали

МАЛЕНЬКИЙ ПРИМЕР · $N = 6$, $U = 4$

СОРТИРОВКА СОБИРАЕТ ПОВТОРЫ РЯДОМ



Вход 7 3 7 1 3 9

sort 1 3 3 7 7 9

unique 1 3 7 9

Один проход оставляет по одному из соседних равных

МАЛЕНЬКИЙ ПРИМЕР · ОДНА ВСТАВКА, ОДНА КОЛЛИЗИЯ, ОДИН ПОВТОР

ХЕШ ВЫБИРАЕТ МЕСТО, РАВЕНСТВО ПРОВЕРЯЕТ КЛЮЧ

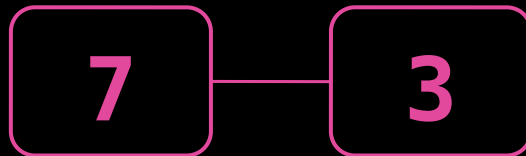


Учебный хеш: $h(x) = x \bmod 4$

7 выбирает бакет 3

3 выбирает тот же бакет

Бакет 3



Вторая 7: ключ уже есть, вставка не нужна

Коллизия: один бакет. Повтор: равные полные ключи.

ЧТО ГОВОРИТ ОЦЕНКА СЛОЖНОСТИ



$O(N \log N)$

Сравниваем ключи

Перемещаем элементы

N — весь вход

U — сколько среди них разных

В среднем $O(N)$

Считаем хеш

Ищем ключ в таблице

Цена сравнения $\neq$ цена вставки

ХЕШ ДОРОЖАЕТ ПОСЛЕ 512K



M2 Pro · libc++ · U = N/2 · нс на элемент



M2 Pro: эта серия Xeon VM: сравнение далее iPhone 14 Pro: сравнение далее

УЗЛЫ СТОЯТ АЛЛОКАЦИЙ И ПРЫЖКОВ



sort + unique

`vector<uint64_t>`



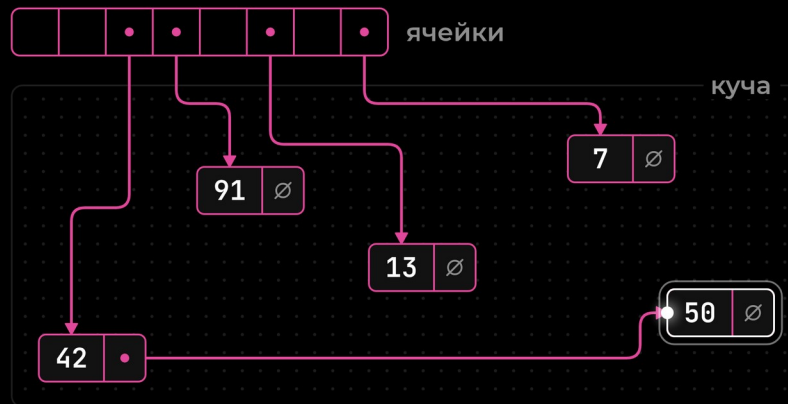
8 Б на ключ, подряд

Узлы: адрес следующего
узнаём после чтения текущего

0 new

Вход готов. $N = U = 2^{20}$, libc++

unordered_set



≈1 млн new

Пример glibc: ≈42 МБ

ЧТО ЛЕЖИТ ВНУТРИ КОЛЛЕКЦИИ



Добавить в конец или вставить в середину?

C++ `vector<uint64_t>`

Числа лежат подряд

Java `ArrayList<Long>`

Ссылки на объекты Long, боксинг

CPython `list[int]`

Указатели на объекты int

Swift `Array<UInt64>`

Числа подряд, COW при общем буфере

Swift `Set<UInt64>` уже использует открытую адресацию

В конец: обычно одна запись, при росте копия буфера

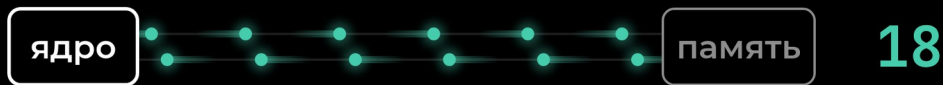
В середину: сдвиг хвоста. Swift при общем буфере добавляет COW

ПРОЦЕССОР МОЖЕТ ЖДАТЬ КАЖДЫЙ ШАГ



Время $\approx N \times \text{шагов на элемент} \times \text{цена шага}$

адреса известны заранее



адрес внутри узла



Независимые чтения перекрываются. Зависимые ждут друг друга

ЗАВИСИМОСТЬ ВИДНА В ИНСТРУКЦИЯХ



Независимые адреса

x86-64

```
mov rax, qword ptr [rdi]
mov rdx, qword ptr [rsi]
```

AArch64

```
ldr x0, [x0]
ldr x1, [x1]
```

Следующий адрес зависит от первой загрузки

x86-64

```
mov rax, qword ptr [rdi]
mov rax, qword ptr [rax]
```

AArch64

```
ldr x8, [x0]
ldr x0, [x8]
```

Слева ожидания можно перекрыть. Справа второй адрес ещё неизвестен

БОЛЬШОЙ L3 НЕ ГАРАНТИРУЕТ БЫСТРЫЙ ШАГ



Паспорт VM

260

МиБ L3 по Iscpu

Зависимое чтение

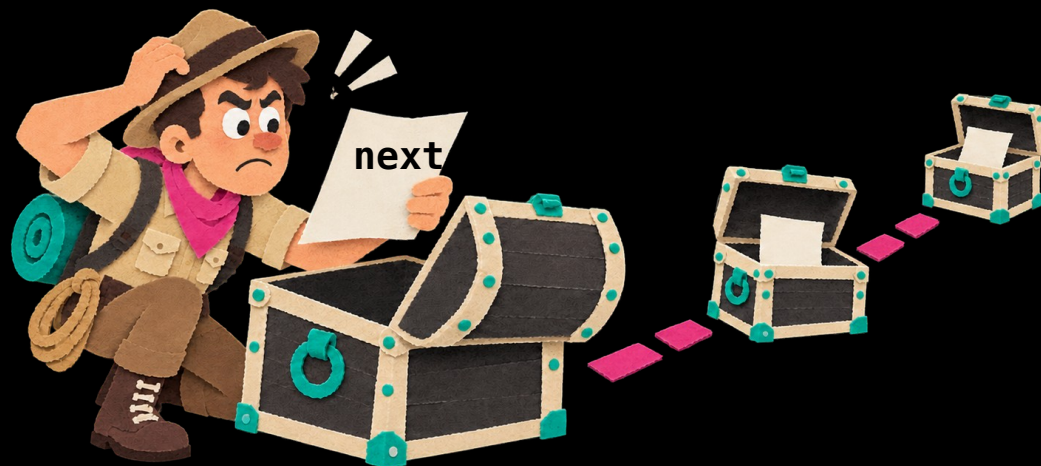
140,8

нс на чтение при наборе 64 МиБ

Паспорт гостя не гарантирует доступную ёмкость и задержку

Доля общего кэша и точный источник чтения здесь неизвестны

ХОТЕЛ ПРОЧИТАТЬ ЧИСЛО

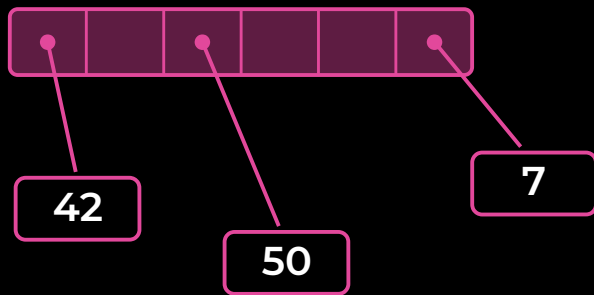


Записался на **квест**

ПЛОСКАЯ ТАБЛИЦА: КЛЮЧИ В МАССИВЕ



unordered_set

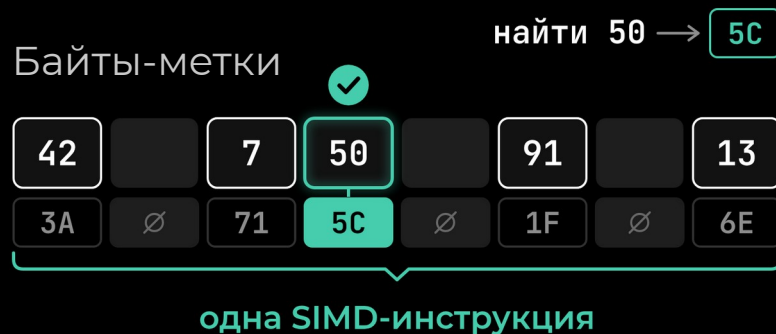


Свой `new` на каждый ключ

Тот же принцип:

`map` в Go ≥ 1.24 , `HashMap` в Rust

flat_hash_set · Swiss Table



- + Одна аллокация на всю таблицу
- Ссылки не переживают рост

ТРИ РЕАЛИЗАЦИИ, ОДИН ВХОД



нс на элемент · $N = U = 2^{20}$ · M2 Pro · libc++ · меньше — лучше

unordered_set



51,3

sort + unique



19,9

flat_hash_set



11,7

×1,7

flat против sort + unique

M2 Pro: эта серия Xeon VM: отдельная серия iPhone 14 Pro: не измерено

В РЕЗЮМЕ У ОБОИХ $O(n)$



`unordered_set`



`flat_hash_set`



СОРТИРОВКА ИЛИ ХЕШ?



M2 Pro · $N = 2^{20}$ · сначала $U = N$ · нс на элемент

sort + unique  20

unordered_set  51

×2,6

Тот же миллион.
Уникальных — 1%.

Кто впереди?

sort

хеш

M2 Pro: эта серия Xeon VM: отдельная серия iPhone 14 Pro: не измерено

ПОБЕДИТЕЛЯ НАЗНАЧАЮТ ДАННЫЕ



нс на элемент · $N = 2^{20}$ · M2 Pro · меньше — лучше

1% уникальных

unordered_set

3,2

sort + unique

12,0

хеш ×3,7

100% уникальных

sort + unique

19,8

sort ×2,6

unordered_set

51,6

M2 Pro: эта серия Xeon VM: отдельная серия iPhone 14 Pro: не измерено

ТРЕБОВАНИЕ В ТИКЕТЕ МЕНЯЕТ ПОБЕДИТЕЛЯ



M2 Pro · libc++ · $N = 2^{20}$ · $U = N/2$ · нс на элемент

Уникальные, порядок любой

sort 20,2 · хеш 22,6

≈ близко

Отсортированные

sort 20,2 · хеш 32,6

sort ×1,6

В порядке первого появления

sort 74,6 · хеш 22,0

хеш ×3,4

«Порядок любой» допускает разный порядок при каждом запуске

M2 Pro: эта серия Xeon VM: не измерено iPhone 14 Pro: не измерено

ПОРЯДОК ТОГО ЖЕ ВХОДА



M2 Pro · libc++ · $N = 2^{20}$ · $U = N/2$ · нс на элемент

Случайный

sort 20,2 · хеш 22,6

≈ близко

Сериями одинаковых

sort 20,5 · хеш 15,5

хеш ×1,3

Уже отсортированный

1

sort 2,0 · хеш 16,5

sort ×8

libc++: ≈2 сравнения вместо ≈22 на элемент

libstdc++ так не делает

M2 Pro: эта серия Xeon VM: отдельная серия iPhone 14 Pro: не измерено

ПРЕДСКАЗАНИЕ ВЕТВЛЕНИЙ



Условие ещё не готово

Процессор предсказывает путь
и начинает выполнять его заранее

Угадал

Работа пригодилась

Ошибся

Часть работы отбросил

История ветвлений и данные в кэше требуют разных контролей

ПОВТОРЯЕМЫЙ ВХОД УСКОРЯЕТ БЕНЧМАРК



sort + unique · N=1024 · нс/элемент

	M2 Pro	Xeon VM	iPhone 14 Pro
Один вход	6,8	9,7	нет серии
64 разных	14,4	46,9	нет серии
Контроль Xeon	64 копии одного входа: 9,6 64 входа через один буфер 8 КиБ: 46,8		

На Xeon важен рисунок входа. Механизм на M2 не установлен

МЕНЯЕМ QoS ПОТОКА, МЕНЯЕТСЯ РЕЗУЛЬТАТ



```
pthread_set_qos_class_self_np(QOS_CLASS_UTILITY, 0);
```

$N = 512K$ · $U = N/2$ · sort / unordered_set, нс на элемент

	userInitiated	utility
M2 Pro libc++	19,6 / 18,7	20,1 / 21,0
Xeon VM Linux	Apple QoS не применим см. сравнение сред	
iPhone 14 Pro A16 Bionic	19,6 / 18,2	55,3 / 105,3

M2 Pro, 1M: преимущество sort выросло с $\times 1,3$ до $\times 1,8$

НА МИЛЛИОНЕ id СОРТИРОВКА ВПЕРЕДИ В 3,3 РАЗА



iPhone 14 Pro · A16 Bionic · QoS utility

×3,3

sort + unique
быстрее unordered_set

$N = 1M$ · $U = N/2$ · одинаковые входы

M2 Pro: см. сравнение Xeon VM: см. сравнение iPhone 14 Pro: utility

нс на элемент · меньше — лучше

sort + unique

57,4



unordered_set

187,7



САМАЯ ВАЖНАЯ ЗАПЯТАЯ



Без меня доклад станет зрелищнее. **И неправильнее**

ТИП ЯДРА МЕНЯЕТ ЦЕНУ ЧТЕНИЯ



P

**Производительное
ядро**

Своя частота и ресурсы

Свои ближайшие кэши

E

**Энергоэффективное
ядро**

Своя частота и ресурсы

Свои ближайшие кэши

QoS сообщает намерение системе

Где выполняется поток, проверяем отдельным опытом

ТИП ЯДРА И ПУТЬ ДАННЫХ



Упрощённый путь чтения при промахх



M2 Pro

L1 ядра
L2 кластера

Xeon VM

L1, L2, L3 видны гостю
Доля кэша неизвестна

iPhone 14 Pro

Свою топологию
проверяем на A16

Большая таблица чаще уходит дальше. Цепочка складывает задержки

«РАБОТАЕТ НА A16»



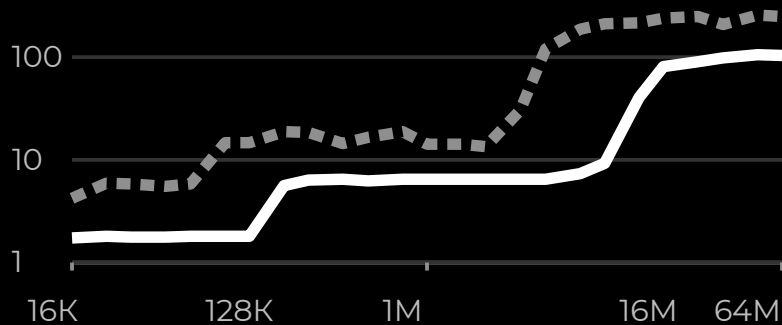
Адрес точный. Но неполный

ИЗЛОМЫ СОГЛАСУЮТСЯ СО СМЕНОЙ ТИПА ЯДРА



M2 Pro

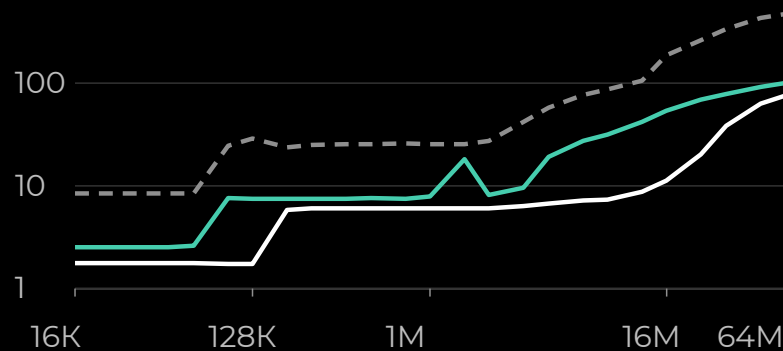
Отдельный pointer-chasing · нс на чтение · КиБ/МиБ · лог. шкалы



userInitiated

background

iPhone 14 Pro · A16



userInitiated

background

utility

Хеон VM: контроль большого L3 уже показали отдельно

ОДИН БЕНЧМАРК, РАЗНЫЕ ПОБЕДИТЕЛИ



sort + unique против unordered_set · $U = N/2$ · × выигрыш по времени

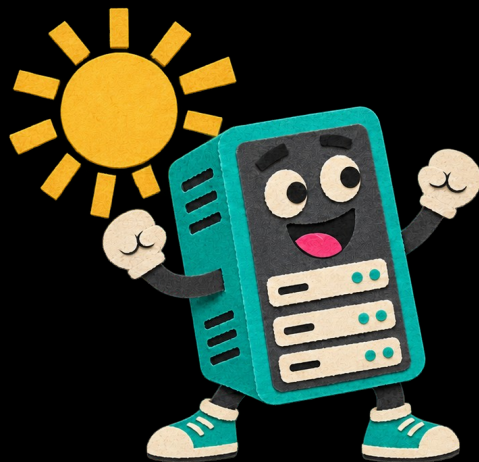
N	M2 Pro userInitiated · libc++	Xeon VM Linux · libstdc++ 13	iPhone 14 Pro A16 Bionic · utility
64K	≈ близко	хеш ×1,64	≈ близко
256K	≈ близко	хеш ×1,29	sort ×1,3
512K	≈ близко	хеш ×1,12	sort ×1,9
1M	sort ×1,3	≈ близко	sort ×3,3

CI отвечает про свою среду. Рейтинг переносим после проверки

CI: «У МЕНЯ ВСЁ ХОРОШО»



CI



iPhone · utility



Прогноз точный. Город другой

ГОТОВЫЙ ПОРЯДОК УЖЕ ЕСТЬ В ДАННЫХ

СЛИЯНИЕ ДВУХ ГОТОВЫХ ПОРЯДКОВ



Локальные	1 3 7
Дельта	3 2 7 2 9
sort + unique дельты	2 3 7 9
Слияние	1 2 3 7 9

Локальную часть повторно не сортируем

СОРТИРОВКА ДЕЛЬТЫ И СЛИЯНИЕ



sort
дельты 64K

unique

слить
с локальными 1M

Уникальные id

```
std::sort(D.begin(), D.end());  
D.erase(std::unique(D.begin(), D.end()), D.end());  
std::set_union(L.begin(), L.end(), D.begin(), D.end(), out);
```

unique убирает дубли дельты

×3,5–6,6

быстрее полной сортировки

Xeon VM · Linux

M2 Pro: не измерено Xeon VM: эта серия iPhone 14 Pro: не измерено

ПРАКТИЧЕСКИЙ ВЫБОР

ЦЕНА В МИЛЛИСЕКУНДАХ



Лабораторная серия · iPhone 14 Pro · QoS utility

512K id · половина уникальных

sort ≈ 29 мс · unordered_set ≈ 55 мс

+26 мс · +12 МиБ

На тестовом 1M: ≈ 60 против ≈ 197 мс

Наш кандидат: sort дельты, unique и слияние

M2 Pro: см. сравнение Xeon VM: см. сравнение iPhone 14 Pro: utility

ПЕРЕНОС НА СВОЮ СРЕДУ



Вся операция, свои данные, целевая конфигурация

M2 Pro

Ядра, режим и libc++

Xeon VM

CPU гостя, библиотека, виртуализация

iPhone 14 Pro

Исполнитель и QoS на устройстве

Коллекция в другом языке требует проверки реализации

КОД-РЕВЬЮ, ВТОРНИК, 17:40



```
// dedup ids after offline sync
- std::sort(ids.begin(), ids.end());
- ids.erase(std::unique(...), ids.end());
+ std::unordered_set<uint64_t> s(...);
+ ids.assign(s.begin(), s.end());
```

Какой
комментарий
вы оставите?

Changes requested

1 Контракт

Какой выход нужен?

2 Данные

Размер, повторы,
порядок входа?

3 Исполнение

CPU, контейнер,
библиотека, QoS?

ВАШ АЛГОРИТМ НЕ ВЫБИРАЛ ПРОЦЕССОР



Проверьте, какой ему достался

Начните с одного перф-теста
на целевом устройстве



github.com/Neoron95/conference-hardfest-algorithms

[Архив для вопросов](#)